

Домашнее задание 2. Настройка CI/CD в Github Actions

Contents

Выбранный проект.....	1
Общая структура CI/CD-пайплайна	2
Код файлов *.yml.....	2
ci.yml.....	2
Код.....	2
Описание.....	5
codeql.yml.....	6
Код.....	6
Описание.....	7
javadoc.yml	8
Код.....	8
Описание.....	9
static-analysis.yml.....	9
Код.....	9
Описание.....	11
ssh-remote-demo.yml.....	12
Код.....	12
Описание.....	13
Ссылка на открытый репозиторий, где настроен pipeline	15

Выбранный проект

В качестве исходного проекта выбран консольный Java-проект на Gradle — CLI-приложение для управления короткими ссылками (ShortLinkApp).

Проект:

- написан на Java 17;
- собирается с помощью Gradle;
- содержит юнит-тесты на JUnit 5;

- использует стандартный Gradle wrapper gradlew / gradlew.bat.

Ссылка на полное описание структуры, архитектуры, CI/CD проекта и т.д. (на англ.)

[shortlinkapp/README.md at master · vkuryndin/shortlinkapp](https://shortlinkapp.com/README.md)

Общая структура CI/CD-пайплайна

Для выбранного проекта CI/CD-пайплайн состоит из следующих workflow:

- ci.yml – основной workflow сборки, автотестов, публикации всех артефактов
- codeql.yml – специфический workflow для запуска статического анализа кода CodeQL
- javadoc.yml – workflow для сборки и публикации Javadoc-документации как на исходный код, так и на тесты
- static-analysis.yml – специализированный workflow для запуска статического анализа кода и проверки форматирования
- ssh-remote-demo.yml – специализированный workflow для эмуляции SSH Remote Commands на GitHub runner (сделано специально для удовлетворения требованиям домашнего задания)

Код файлов *.yml

ci.yml

Код

name: CI

on:

```
push:      # Run CI on every push to any branch
pull_request: # and on every pull request (for pre-merge checks)
```

jobs:

```
build:
  runs-on: ${{ matrix.os }}
```

permissions:

```
  contents: write # Needed to commit updated coverage badges back to the repo
```

strategy:

```
  fail-fast: false
  matrix:
    os:
      - ubuntu-latest      # Linux
      - windows-latest    # Windows
      - macos-14           # macOS Apple Silicon (arm64)
      - macos-15-intel     # macOS Intel (x86_64)
```

steps:

```
# 1) Checkout the repository code
- name: Checkout
  uses: actions/checkout@v4

# 2) Install JDK 17 (Temurin distribution)
- name: Set up JDK 17
  uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: '17'
```

```

# 3) Configure Gradle with caching (official Gradle GitHub Action)
- name: Setup Gradle (with caching)
  uses: gradle/actions/setup-gradle@v4

# 4) Make Gradle wrapper executable (required on Linux/macOS)
- name: Make Gradle wrapper executable
  run: chmod +x gradlew

# 5a) Main CI step for Linux/macOS: full Gradle build
- name: Build (tests + coverage gate) [Linux/macOS]
  if: runner.os != 'Windows'
  run: ./gradlew clean build --no-daemon --stacktrace

# 5b) Main CI step for Windows: use gradlew.bat
- name: Build (tests + coverage gate) [Windows]
  if: runner.os == 'Windows'
  run: .\gradlew.bat clean build --no-daemon --stacktrace

# 6a) Build fat JAR (all dependencies) using Shadow plugin on Linux/macOS
- name: Build fat JAR (all dependencies) [Linux/macOS]
  if: runner.os != 'Windows'
  run: ./gradlew shadowJar --no-daemon --stacktrace

# 6b) Build fat JAR (all dependencies) on Windows
- name: Build fat JAR (all dependencies) [Windows]
  if: runner.os == 'Windows'
  run: .\gradlew.bat shadowJar --no-daemon --stacktrace

# 7) Compute a safe, unique artifact name based on branch name and run number
- name: Compute safe artifact name
  id: name
  shell: bash
  env:
    MATRIX_OS: ${matrix.os}
  run: |
    ref="${GITHUB_HEAD_REF:-${GITHUB_REF_NAME}}"
    ref="${ref//\//-}" # replace "/" with "-"
    os="${MATRIX_OS//\//-}" # sanitize OS label
    echo "safe_name=shortlinkapp-${ref}-${os}-${GITHUB_RUN_NUMBER}" >> "$GITHUB_OUTPUT"

# 8) "Deploy" the built JAR artifacts (we upload all compiled artifacts: simple and fat
with all dependencies):
# Upload the JAR produced by the build so it can be downloaded from the Actions UI.
- name: Upload build artifacts (JAR)
  if: success() # Only if the build finished successfully
  uses: actions/upload-artifact@v4
  with:
    name: ${steps.name.outputs.safe_name}
    path: '**/build/libs/*.jar' # All JARs under any build/libs directory
    if-no-files-found: warn
    retention-days: 7 # Keep artifact for 7 days

# 9) Run the CLI fat JAR on the GitHub runner (simple smoke test)
# We feed 'q' to stdin to exit gracefully after showing the menu.
# We only do this on Linux to avoid duplicating the same check on all OSes.
- name: Run CLI smoke test on GitHub runner
  if: success() && matrix.os == 'ubuntu-latest' # previous -if: success() # run only
if build & artifacts upload succeeded
  shell: bash
  run: |
    set -e

    JAR=$(ls -t build/libs/*-all.jar | head -n 1)

    if [[ -z "$JAR" ]]; then

```

```

    echo "Fat JAR (*-all.jar) not found in build/libs"
    exit 1
fi

echo "Using fat JAR: $JAR"

# Capture output of the CLI
OUTPUT=$(printf 'q\n' | java -jar "$JAR")

# Print output to the GitHub Actions log
echo "----- CLI output start -----"
echo "$OUTPUT"
echo "----- CLI output end -----"

# Simple assertion: check for banner line
echo "$OUTPUT" | grep -q "ShortLink CLI (Java)" || {
    echo "Expected banner 'ShortLink CLI (Java)' not found in output"
    exit 1
}

echo "CLI smoke test finished successfully."
# 10) Upload JUnit test reports (HTML) for inspection
# Always attaching report, even if build fails
- name: junit-test-report-${{ matrix.os }}      #fixing multiply OS
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: junit-test-report-${{ matrix.os }}
    path: build/reports/tests/test
    if-no-files-found: ignore
    retention-days: 7

# 11) Upload JaCoCo HTML coverage report
- name: Upload JaCoCo HTML report
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: jacoco-html-${{ matrix.os }}          #fixing multiply OS
    path: build/reports/jacoco/test/html
    if-no-files-found: warn
    retention-days: 7

# 12) Upload SpotBugs reports (static analysis)
- name: Upload SpotBugs reports
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: spotbugs-reports-${{ matrix.os }}    #fixing multiply OS
    path: |
      build/reports/spotbugs/main.html
      build/reports/spotbugs/test.html
    if-no-files-found: ignore
    retention-days: 7

# 13) Upload Checkstyle reports (code style violations)
- name: Upload Checkstyle reports
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: checkstyle-reports-${{ matrix.os }}  #fixing multiply OS
    path: build/reports/checkstyle
    if-no-files-found: ignore
    retention-days: 7

# 14) Generate JaCoCo SVG badges (line coverage + branch coverage)

```

```

# Only for non-PR builds (we do not want to commit from pull requests).
- name: Generate JaCoCo badges
  if: github.event_name != 'pull_request' && matrix.os == 'ubuntu-latest' #will do only
on Linux
  uses: cicirello/jacoco-badge-generator@v2.12.1
  with:
    jacoco-csv-file: build/reports/jacoco/test/jacocoTestReport.csv
    badges-directory: badges
    coverage-badge-filename: jacoco.svg
    branches-badge-filename: branches.svg
    generate-coverage-badge: true
    generate-branches-badge: true
    # optionally, for the build not to fail, if CSV not found:
    # on-missing-report: warn

# 15) Commit and push the updated coverage badges back to the repository
- name: Commit badges
  if: github.event_name != 'pull_request' && matrix.os == 'ubuntu-latest' #will do only
on Linux
  uses: EndBug/add-and-commit@v9
  with:
    add: "badges/*.svg"
    message: "chore(ci): update coverage badge"
    default_author: github_actions

```

Описание

Build – основной job, который содержит все необходимые шаги. Запускается на разных операционных системах .

```

matrix:
  os:
    - ubuntu-latest      # Linux
    - windows-latest     # Windows
    - macos-14           # macOS Apple Silicon (arm64)
    - macos-15-intel     # macOS Intel (x86_64)

```

Имеет разрешение на запись, так как нам надо загрузить обновленные SVG-бэджи статуса сборки и покрытия кода в репозиторий после всех тестов.

```

permissions:
  contents: write # Needed to commit updated coverage badges back to the repo

```

Содержит следующие шаги:

1. Выгрузка кода репозитория
2. Установка JDK 17
3. Настройка Gradle с кэшированием
4. Настройка флага executable для gradle wrapper, так как это требуется на Linux/macOS
5. Сборка кода, обычный jar (без зависимостей)
 - a. Сборка под Linux/macOS
 - b. Сборка под Windows
6. Сборка fat jar (jar с зависимостями через ShadowJar)
7. Создание уникального имени артефактов, чтобы не работало на разных ОС , учитывая ветки
8. Загрузка полученных артефактов сборки (jar-файлы) (чтобы были доступны на странице actions/run в GitHub). Пример [fixng SSH Remote commands · vkuryndin/shortlinkapp@57f0ae7](#)
9. Запуск CLI на GitHub Runner, чтобы убедиться, что приложение запускается

10. Загрузка отчетов тестов Junit (чтобы были доступны на странице actions/run в GitHub).
11. Загрузка артефактов покрытия кода Јососо (чтобы были доступны на странице actions/run в GitHub).
12. Загрузка артефактов статического анализа SpotBugs (чтобы были доступны на странице actions/run в GitHub).
13. Загрузка отчетов проверки стиля Checkstyle (чтобы были доступны на странице actions/run в GitHub)
14. Создание SVG-баджей, показывающих статус build-а и процент покрытия кода тестами, чтобы потом вставить их в readme.md ([shortlinkapp/README.md at master · vkuryndin/shortlinkapp](#))
15. Комит загруженных на этапе выше SVG-баджей в репозиторий (чтобы они отображались в readme.md) ([shortlinkapp/README.md at master · vkuryndin/shortlinkapp](#))

Подробнее обо всех запусках см [CI · Workflow runs · vkuryndin/shortlinkapp](#)

codeql.yml

Код

name: CodeQL

on:

```
push:
  branches: ["**"]      # Run CodeQL on push to any branch
  paths-ignore:
    - ".github/workflows/**" # do NOT trigger when only workflow files change
pull_request:
  branches: ["**"]      # And on all pull requests
  paths-ignore:
    - ".github/workflows/**" # do NOT trigger when only workflow files change
schedule:
  - cron: "17 3 * * 1"  # Weekly scheduled scan (every Monday at 03:17 UTC)
workflow_dispatch:      # Manual trigger from GitHub UI
```

permissions:

```
contents: read          # Read access to the repository
security-events: write  # Required to upload CodeQL analysis results to the Security tab
actions: read           # Recommended for CodeQL
```

jobs:

```
analyze:
  name: CodeQL Static Analysis
  runs-on: ubuntu-latest
  timeout-minutes: 60

  strategy:
    fail-fast: false
    matrix:
      language: ["java"]  # Analyze Java code
```

steps:

```
# 1) Checkout the repository
- name: Checkout
  uses: actions/checkout@v4

# 2) Initialize CodeQL with the chosen language and query suite
- name: Initialize CodeQL
  uses: github/codeql-action/init@v3
  with:
    languages: ${matrix.language }
```

```

        queries: security-and-quality

# 3) Autobuild step:
#   For many Java projects CodeQL can build the project automatically.
- name: Autobuild
  uses: github/codeql-action/autobuild@v3

# For debugging purposes:
# If autobuild fails for any reason, uncomment this block and comment out the autobuild
step.
# - name: Manual Gradle build
#   run: |
#     chmod +x gradlew
#     ./gradlew clean build --no-daemon --stacktrace

# 4) Run CodeQL analysis and upload results
- name: Perform CodeQL Analysis
  uses: github/codeql-action/analyze@v3
  with:
    category: "/language:${{ matrix.language }}" #Java is used here

```

Описание

Сначала мы настраиваем критерии запуска.

```

on:
  push:
    branches: [ "**" ] # Run CodeQL on push to any branch
    paths-ignore:
      - ".github/workflows/**" # do NOT trigger when only workflow files change
  pull_request:
    branches: [ "**" ] # And on all pull requests
    paths-ignore:
      - ".github/workflows/**" # do NOT trigger when only workflow files change
  schedule:
    - cron: "17 3 * * 1" # Weekly scheduled scan (every Monday at 03:17 UTC)
  workflow_dispatch: # Manual trigger from GitHub UI

```

Далее настраиваем разрешения

```

permissions:
  contents: read # Read access to the repository
  security-events: write # Required to upload CodeQL analysis results to the Security tab
  actions: read # Recommended for CodeQL

```

Analyze – основной job, в котором мы делаем статический анализ исходного кода с помощью CodeQL.

Состоит из следующих шагов:

1. Выгрузка исходного кода из репозитория
2. Инициализация CodeQL с языком программирования java и настройками проверки – security-and-quality
3. Сборка кода
4. Запуск статического анализа CodeQL и выгрузка результатов

Подробнее смотри [CodeQL · Workflow runs · vkuryndin/shortlinkapp](#) и [trying to fix test om multiply OS · vkuryndin/shortlinkapp@626ed46](#)

javadoc.yml

Специально сделал в рамках обучения для сборки и публикации Javadoc-документации.

Код

```
name: Javadoc

on:
  push:
    branches: [ main, master, develop ] # Generate Javadoc only for main development branches
    paths:
      - '**/*.java' # Trigger only when Java sources change
      - 'build.gradle*'
      - 'settings.gradle*'
    workflow_dispatch: # Manual trigger from GitHub UI

permissions:
  contents: read

jobs:
  javadoc:
    runs-on: ubuntu-latest
    steps:
      # 1) Checkout the repository
      - name: Checkout
        uses: actions/checkout@v4

      # 2) Install JDK 17 (Temurin distribution)
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'

      # 3) Configure Gradle with caching (official Gradle GitHub Action)
      - name: Setup Gradle (with caching)
        uses: gradle/actions/setup-gradle@v4

      # 4) Make Gradle wrapper executable (required on Linux/macOS)
      - name: Make Gradle wrapper executable
        run: chmod +x gradlew

      # 5) Generate Javadoc for main sources
      - name: Generate Javadoc (main)
        run: ./gradlew javadoc --no-daemon

      # 6) Generate Javadoc for tests (with Xdoclint:none to be more tolerant)
      - name: Generate Javadoc (tests with Xdoclint:none)
        run: ./gradlew testJavadoc --no-daemon

      # 7) Upload main Javadoc as a build artifact (acts as "documentation deployment")
      - name: Upload Javadoc artifact (main)
        uses: actions/upload-artifact@v4
        with:
          name: javadoc
          path: build/docs/javadoc
          if-no-files-found: error
          retention-days: 14 # Keep Javadoc for 14 days

      # 8) Upload test Javadoc as a separate artifact
      - name: Upload Javadoc artifact (tests)
        uses: actions/upload-artifact@v4
        with:
```



```
name: javadoc-tests
path: build/docs/testJavadoc
if-no-files-found: error
retention-days: 14 # Keep Javadoc for 14 days
```

Описание

Сначала мы настраиваем критерии запуска.

```
on:
  push:
    branches: [ main, master, develop ] # Generate Javadoc only for main development branches
    paths:
      - '**/*.java' # Trigger only when Java sources change
      - 'build.gradle*'
      - 'settings.gradle*'
  workflow_dispatch: # Manual trigger from GitHub UI
```

Javadoc - основной job, состоящий из следующих шагов:

1. Выгрузка исходного кода из репозитория
2. Установка JDK 17
3. Настройка Gradle с кэшированием
4. Настройка флага executable для gradle wrapper, так как это требуется на Linux/macOS
5. Создание Javadoc-документации для основных исходных кодов (org.example.main.java...)
6. Создание Javadoc-документации для исходных кодов тестов (org.example.test.java...)
7. Загрузка артефактов Javadoc-документации для основных исходных кодов, чтобы они были доступны с помощью GitHub UI (документация к каждому билду хранится 14 дней) (например, см тут [trying to fix test om multiply OS · vkuryndin/shortlinkapp@c645b3d](#))
8. Загрузка артефактов Javadoc-документации для исходных кодов тестов, чтобы они были доступны с помощью GitHub UI (документация к каждому билду хранится 14 дней) (например, см тут [trying to fix test om multiply OS · vkuryndin/shortlinkapp@c645b3d](#))

Подробнее см [Javadoc · Workflow runs · vkuryndin/shortlinkapp](#)

static-analysis.yml

Код

```
name: Static Analysis

on:
  push:
    branches: [ main, master, develop, feature/** ] # Run on main branches and feature branches
    paths:
      - '**/*.java'
      - 'build.gradle'
      - 'build.gradle.kts'
      - 'settings.gradle'
      - 'settings.gradle.kts'
      - 'gradle.properties'
      - 'config/checkstyle/**'
      - '.github/workflows/static-analysis.yml'
  pull_request:
    paths:
      - '**/*.java'
      - 'build.gradle'
      - 'build.gradle.kts'
      - 'settings.gradle'
      - 'settings.gradle.kts'
```

```

    - 'gradle.properties'
    - 'config/checkstyle/**'
    - '.github/workflows/static-analysis.yml'

permissions:
  contents: read

# Ensure only one static-analysis run per branch is active at a time
concurrency:
  group: static-analysis-${{ github.ref }}
  cancel-in-progress: true

jobs:
  static-analysis:
    name: Spotless · Checkstyle · SpotBugs
    runs-on: ubuntu-latest

    steps:
      # 1) Checkout the repository
      - name: Checkout
        uses: actions/checkout@v4

      # 2) Install JDK 17 (Temurin distribution)
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'

      # 3) Configure Gradle with caching (official Gradle GitHub Action)
      - name: Setup Gradle (with caching)
        uses: gradle/actions/setup-gradle@v4

      # 4) Make Gradle wrapper executable (required on Linux/macOS)
      - name: Make Gradle wrapper executable
        run: chmod +x gradlew

      # 5) Run only static-analysis tasks (no compilation tests or packaging beyond what they
need)
      - name: Run static checks (Spotless, Checkstyle, SpotBugs)
        run: |
          ./gradlew \
            spotlessCheck \
            checkstyleMain checkstyleTest \
            spotbugsMain spotbugsTest \
            --no-daemon --stacktrace

      # 6) Upload Spotless diff files (if any formatting issues detected)
      # Always publish the HTML reports to help debugging even on failures
      - name: Upload Spotless diff (if any)
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: spotless-diff
          path: '**/.gradle/spotless/spotless*.diff'
          if-no-files-found: ignore
          retention-days: 7

      # 7) Upload Checkstyle HTML/XML reports
      - name: Upload Checkstyle reports
        if: always()
        uses: actions/upload-artifact@v4
        with:

```

```

    name: checkstyle-reports
    path: build/reports/checkstyle
    if-no-files-found: ignore
    retention-days: 7

# 8) Upload SpotBugs reports (HTML + XML)
- name: Upload SpotBugs reports
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: spotbugs-reports
    path: |
      build/reports/spotbugs/main.html
      build/reports/spotbugs/test.html
      build/reports/spotbugs/main.xml
      build/reports/spotbugs/test.xml
    if-no-files-found: ignore
    retention-days: 7

# 9) write a brief summary to the job summary page
- name: Summary
  if: always()
  run: |
    echo "## Static Analysis Summary" >> $GITHUB_STEP_SUMMARY
    echo "- Spotless: ran `spotlessCheck`" >> $GITHUB_STEP_SUMMARY
    echo "- Checkstyle: reports saved under `build/reports/checkstyle`" >>
$GITHUB_STEP_SUMMARY
    echo "- SpotBugs: HTML/XML reports uploaded as artifacts" >> $GITHUB_STEP_SUMMARY

```

Описание

Основной CI-пайплайн (ci.yml) делает всё, статический анализ включён в сборку. Дополнительно я создал узкоспециализированный workflow Static Analysis, который запускает только Spotless/Checkstyle/SpotBugs на Linux и публикует отчёты отдельно, чтобы удобнее анализировать качество кода.

В начале, как обычно, настраиваем пути, условия запуска и устанавливаем разрешения.

```

on:
  push:
    branches: [ main, master, develop, feature/** ] # Run on main branches and feature branches
    paths:
      - '**/*.java'
      - 'build.gradle'
      - 'build.gradle.kts'
      - 'settings.gradle'
      - 'settings.gradle.kts'
      - 'gradle.properties'
      - 'config/checkstyle/**'
      - '.github/workflows/static-analysis.yml'
  pull_request:
    paths:
      - '**/*.java'
      - 'build.gradle'
      - 'build.gradle.kts'
      - 'settings.gradle'
      - 'settings.gradle.kts'
      - 'gradle.properties'
      - 'config/checkstyle/**'
      - '.github/workflows/static-analysis.yml'

permissions:
  contents: read

```

Основной job – static-analysis, запускается только на Linux. Состоит из следующих шагов:

1. Выгрузка исходного кода из репозитория
2. Установка JDK 17
3. Настройка Gradle с кэшированием
4. Настройка флага executable для gradle wrapper, так как это требуется на Linux/macOS
5. Запуск spotlessCheck, checkstyleMain/checkstyleTest, spotbugsMain/spotbugsTest (для основной кодовой базы и автотестов)
6. Загрузка результатов работы spotless, чтобы было видно в интерфейсе GitHub actions
7. Загрузка отчетов Checkstyle, чтобы было видно в интерфейсе GitHub actions
8. Загрузка результатов SpotBugs, , чтобы было видно в интерфейсе GitHub actions
9. Запись общей информации о работе job-а на страницу (например, см [trying to fix test on multiply OS · vkuryndin/shortlinkapp@c645b3d](#)).

Поробнее см [Static Analysis · Workflow runs · vkuryndin/shortlinkapp](#)

ssh-remote-demo.yml

Код

name: SSH Remote Commands Demo

on:

```
# Manual trigger only: run from GitHub UI (Actions -> this workflow -> Run workflow)
workflow_dispatch:
```

jobs:

ssh-remote-demo:

name: Build & run via SSH on localhost
runs-on: ubuntu-latest #only on ubuntu

steps:

```
# 1) Checkout the repository code
- name: Checkout
  uses: actions/checkout@v4
```

```
# 2) Install JDK 17 (Temurin distribution)
- name: Set up JDK 17
  uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: '17'
```

```
# 3) Configure Gradle with caching (official Gradle GitHub Action)
- name: Setup Gradle (with caching)
  uses: gradle/actions/setup-gradle@v4
```

```
# 4) Make Gradle wrapper executable (required on Linux/macOS)
- name: Make Gradle wrapper executable
  run: chmod +x gradlew
```

```
# 5) Build the application and fat JAR (with all dependencies)
- name: Build app (tests + fat JAR)
  run: ./gradlew clean build shadowJar --no-daemon --stacktrace
```

```
# 6) Install and start SSH server on the GitHub runner
# This simulates a "remote host" reachable via SSH.
- name: Install and start SSH server (localhost)
  run: |
    set -e
```

```

# Install OpenSSH server if it's not present
sudo apt-get update
sudo apt-get install -y openssh-server

# Ensure the runtime directory exists and start sshd
sudo mkdir -p /run/sshd
sudo service ssh start

# Generate a temporary SSH key pair for the current runner user
mkdir -p ~/.ssh
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -N "" -q

# Authorize this key for localhost logins
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys

# 7) Run the CLI application via SSH (remote command on localhost)
- name: Run CLI via SSH on localhost
  shell: bash
  run: |
    set -e

    echo "Local runner info:"
    echo "  pwd: $(pwd)"
    echo "  contents of build/libs:"
    ls -l build/libs || echo "build/libs not found"

    # Find the latest fat JAR (*-all.jar) built by Shadow (relative path)
    JAR_REL=$(ls -t build/libs/*-all.jar | head -n 1 || true)

    if [[ -z "$JAR_REL" ]]; then
      echo "Fat JAR (*-all.jar) not found in build/libs"
      exit 1
    fi

    # Convert to absolute path so SSH session can see the same file
    JAR="$(cd "$(dirname "$JAR_REL")" && pwd)/$(basename "$JAR_REL")"

    echo "Using fat JAR (absolute path): $JAR"

    # Execute the CLI on the "remote" host via SSH.
    # We pass "q" to stdin so the app prints banner/menu and exits.
    OUTPUT=$(ssh -i ~/.ssh/id_rsa -o StrictHostKeyChecking=no localhost \
      "echo 'Remote PWD:' \$(pwd); \
      ls -l '$JAR' || echo 'JAR not visible on remote FS'; \
      printf '4\nq\n' | java -jar '$JAR'")

    echo "----- SSH CLI output start -----"
    echo "$OUTPUT"
    echo "----- SSH CLI output end -----"

    # Simple assertion: banner must be present in the output
    echo "$OUTPUT" | grep -q "ShortLink CLI (Java)" || {
      echo "Expected banner 'ShortLink CLI (Java)' not found in output"
      exit 1
    }

    echo "SSH-based CLI smoke test (documentation + exit) finished successfully."

```

Описание

ssh-remote-demo – специальный пайплайн, созданный для эмуляции SSH remote commands для удовлетворения требованиям задания. Насколько я понял, хостинги не дают бесплатных серверов

для данных задач + настройка удаленного сервера требует времени, которого не хватает. Если это критично для получения максимального балла, прошу сообщить явно.

ssh-remote-demo – основной job этого пайплайна, запускается только под Linux. Данный пайплайн настроен так, чтобы запускать его можно было только вручную, с этой страницы [SSH Remote Commands Demo · Workflow runs · vkuryndin/shortlinkapp](#).

Пайплайн состоит из следующих шагов:

1. Выгрузка исходного кода из репозитория
2. Установка JDK 17
3. Настройка Gradle с кэшированием
4. Настройка флага executable для gradle wrapper, так как это требуется на Linux/macOS
5. Сборка приложения (fat jar) со всеми зависимостями.
6. Установка и запуск SSH сервера на Github Runner. Это эмулирует требование из задания по запуску на удаленном узле.
7. Запуск моего консольного Java- приложения с помощью SSH. При запуске я вывожу в консоль первое меню приложения, потом в консоли автоматически нажимаю 4, чтобы показать внутреннюю документацию по приложению, потом нажимаю q, чтобы корректно выйти из приложения.

Пример запуска

The screenshot shows a GitHub Actions workflow run for the repository 'vkuryndin/shortlinkapp'. The specific job is 'Build & run via SSH on localhost', which succeeded yesterday in 1m 28s. The job steps are:

- Build app (tests + fat JAR)
- Install and start SSH server (localhost)
- Run CLI via SSH on localhost

The output of the 'Run CLI via SSH on localhost' step is shown below, displaying the terminal output of the application running on the remote host:

```
1 ▶ Run set -e
51 Local runner info:
52   pwd: /home/runner/work/shortlinkapp/shortlinkapp
53   contents of build/libs:
54   total 388
55 -rw-r--r-- 1 runner runner 336196 Nov 25 10:54 shortlinkapp-1.0-SNAPSHOT-all.jar
56 -rw-r--r-- 1 runner runner 55837 Nov 25 10:54 shortlinkapp-1.0-SNAPSHOT.jar
57 Using fat JAR (absolute path): /home/runner/work/shortlinkapp/shortlinkapp/build/libs/shortlinkapp-1.0-SNAPSHOT-all.jar
58 Warning: Permanently added 'localhost' (EO25519) to the list of known hosts.
59 ----- SSH CLI output start -----
60 Remote PWD: /home/runner
61 -rw-r--r-- 1 runner runner 336196 Nov 25 10:54 /home/runner/work/shortlinkapp/shortlinkapp/build/libs/shortlinkapp-1.0-SNAPSHOT-all.jar
62 =====
63 ShortLink CLI (Java)
64 =====
65 User UUID: 0aca089b-59b8-4594-bfd4-a10d9b982f30
66 Config loaded from: /home/runner/data/config.json
67 Type a number to choose an option, 'q' to quit.
68
69
70 Main Menu
71 1. My Links
72 2. Open Short Link
73 3. Maintenance
74 4. Help / Examples
75 5. Settings (from config.json)
76 6. Users
77 7. Exit
78 Select:
79 Help / Examples
80 - Only valid http/https URLs are accepted (must contain a host).
81 - Short link format: cli://<shortCode> (you may enter just <shortCode>).
82 - TTL and default click limit are loaded from data/config.json.
83 - Expired links become EXPIRED and cannot be opened.
84 - After reaching the click limit, a link becomes LIMIT_REACHED.
85 - Only the owner can edit or delete a link.
```

Ссылка на открытый репозиторий, где настроен pipeline
[vkuryndin/shortlinkapp: initial commit](https://github.com/vkuryndin/shortlinkapp)